

VHDL Assignment #3: Adders and Critical Path

1 Instructions

- TA in charge: Maninder Bir Singh Gulshan (maninder.gulshan@mail.mcgill.ca) - please utilize discussion boards on myCourses for questions as much as possible.
- Due date: Tuesday, October 24, 2023 by 11:59 pm EDT.
- Submission is in teams using myCourses (only one team member submits). In the report, provide the names and McGill IDs of the team members.
- Late submissions will incur penalties as described in the course syllabus.

2 Learning Outcomes

After completing this assignment you should know how to:

- Use CAD tools and VHDL to implement logic functions, specifically, ripple-carry and BCD adders.
- Synthesize logic functions
- Perform functional simulation
- Use Quartus Timing Analyzer tools to find the critical path

3 Introduction

In this assignment, you will build upon previously implemented circuits to design a more complex circuit. You will learn how to design and simulate useful adder circuit blocks.

If you need any help regarding the lab materials, you can

- Ask the TA for help during lab sessions and office hours.
- Refer to the text book. In case you are not aware, Appendix A “VHDL Reference” provides detailed information on VHDL.
- You can also refer to the tutorial on Quartus and ModelSim provided by Intel ([click here for Quartus](#) and [here for ModelSim](#)).

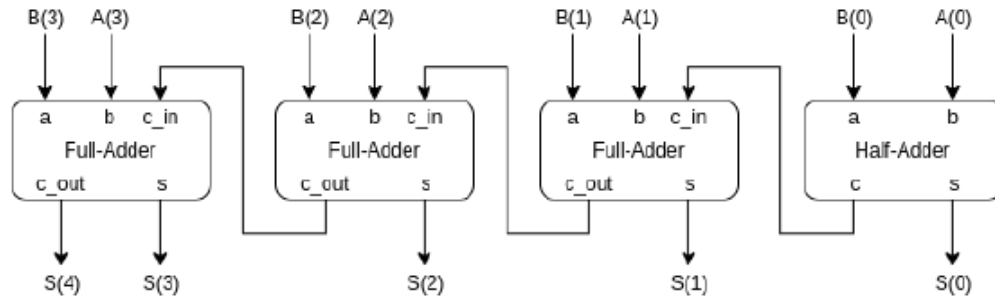
It is highly recommended that you first try to resolve any issue by yourself (refer to the textbook and/or the multitude of VHDL resources on the Internet). Syntax errors, especially, can be quickly resolved by reading the error message to see exactly where the error occurred and checking the VHDL Reference or examples in the textbook for the correct syntax.

4 VHDL Description of Adder Circuits

In this section, you will be asked to perform the design and simulation of the following two adder circuits: (a) a 4-bit ripple-carry adder; and (b) a one-digit binary coded decimal (BCD) adder. Details of the assignments are described below.

4.1 Ripple-Carry Adder (RCA)

In this section, you will implement a structural description of a 4-bit ripple-carry adder using basic addition components: half-adders and full-adders.



4.1.1 Structural Description of a Half-Adder in VHDL

A half-adder is a circuit that takes two binary digits as inputs, and produces the result of the addition of the two bits in the form of a sum and carry signals. The carry signal represents an overflow into the next digit of a multi-digit addition. Using the following entity definition for your VHDL code, implement a structural description of the half-adder.

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity half_adder is
  port (a: in std_logic;
        b: in std_logic;
        s: out std_logic;
        c: out std_logic);
end half_adder;

```

After you have described your structural style of the half-adder in VHDL, you are required to test your circuit. Write a testbench code and perform an exhaustive test of your VHDL description of the half-adder.

4.1.2 Structural Description of a Full-Adder in VHDL

Unlike the half-adder, a full-adder adds binary digits while accounting for values carried in (from a previous stage addition). Write a structural VHDL description for the full-adder circuit using the half-adder circuit that you designed in the previous section. Use the following entity declaration for your structural VHDL description of the full-adder.

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity full_adder is
  port (a: in std_logic;
        b: in std_logic;
        cin: in std_logic;
        s: out std_logic;
        cout: out std_logic);
end full_adder;

```

After you have described your circuit in VHDL, write a testbench code and perform an exhaustive test of your VHDL description of the full-adder.

4.1.3 Structural Description of a 4-bit Ripple-Carry Adder (RCA) in VHDL

Using the half-adder and full-adder circuits implemented in the two previous sections, implement a 4-bit carry-ripple adder. Write a structural VHDL code for the 4-bit RCA using the following entity declaration.

```

library IEEE;

```

```

use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity rca_structural is
port ( A: in std_logic_vector (3 downto 0);
      B: in std_logic_vector (3 downto 0);
      S: out std_logic_vector (4 downto 0));
end rca_structural;

```

Note that $S(4)$ contains the carry-out of the 4-bit adder. After you have described your circuit in VHDL, write a testbench code and perform an exhaustive test of your VHDL structural description of the 4-bit RCA.

4.1.4 Behavioral Description of a 4-bit RCA in VHDL

In this part, you are required to implement the 4-bit RCA using behavioral description. One way to obtain a behavioral description is to use arithmetic operators in VHDL (*i.e.*, “+”). Write a behavioral VHDL code for the 4-bit RCA using the following entity declaration for your behavioral VHDL description.

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity rca_behavioral is
port ( A: in std_logic_vector (3 downto 0);
      B: in std_logic_vector (3 downto 0);
      S: out std_logic_vector (4 downto 0));
end rca_behavioral;

```

After you have described your circuit in VHDL, write a testbench code and perform an exhaustive test of your VHDL behavioral description of the 4-bit RCA.

4.2 VHDL Description of a One-Digit BCD Adder

In this section, you will implement a one-digit BCD adder in VHDL. A one-digit BCD adder adds two four-bit numbers represented in a BCD format. The result of the addition is a BCD-format 4-bit output, representing the decimal sum, and a carry that is generated if this sum exceeds a decimal value of 9 (see slides of Lecture #11).

4.2.1 Structural Description of a BCD Adder in VHDL

In this part, you are required to implement the BCD adder using structural description. You are allowed to use either behavioral or structural style of coding for your implementation. Using the following entity definition. Use the following entity declaration.

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity bcd_adder_structural is
port ( A: in std_logic_vector (3 downto 0);
      B: in std_logic_vector (3 downto 0);
      S: out std_logic_vector (3 downto 0);
      C: out std_logic);
end bcd_adder_structural;

```

After you have implemented the one-digit BCD adder in VHDL, you are required to test your circuit. Write a testbench code and perform an exhaustive test of your VHDL structural description of the one-digit BCD adder.

4.2.2 Behavioral Description of a BCD Adder in VHDL

In this part, you are required to implement the BCD adder using behavioral description. You are encouraged to base your code on the VHDL code in Section 5.7.3 of the textbook, so that you learn about conditional signal assignments (these are explained in detail in the same section as well as in the Appendix in Section A.7.4). Use the following entity declaration.

```

library IEEE;
use IEEE.STD_LOGIC_11164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity bcd_adder_behavioral is
port ( A: in std_logic_vector (3 downto 0);
      B: in std_logic_vector (3 downto 0);
      S: out std_logic_vector (3 downto 0);
      C: out std_logic);
end bcd_adder_behavioral;

```

After you have implemented the one-digit BCD adder in VHDL, you are required to test your circuit. Write a testbench code and perform an exhaustive test for your VHDL behavioral description of the one-digit BCD adder.

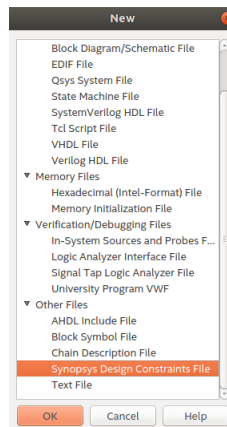
5 Critical Path of Digital Circuits

In this part, you will learn how to use the Quartus CAD tool to determine the delay of a given path in digital circuits. To this end, in this section, we use the ripple-carry adder circuit (that you designed in VHDL assignment #3) as the “circuit under examination”.

Follow the instructions described in VHDL Assignment #1 to create a project. Make sure to select the **Cyclone V** family of FPGAs, with the following part number: **5CSEMA5F31C6** when creating a project. Once created, import the VHDL description of your digital circuit into the project and compile it to make sure there are no syntax errors in your design.

The critical path is the longest path in the circuit and limits the speed of the circuit speed. The speed of a digital circuit is measured in terms of latency and throughput. Latency is the time needed for the circuit to produce an output for a given input (*i.e.*, the total propagation delay (time) from the input to the output), and it is expressed units of time. Alternatively, throughput refers to the rate at which data can be processed. In this assignment, we only consider the latency as a metric to measure the speed of the circuit. In general, digital circuits are subject to timing constraints dictated by the target application. Whether a circuit meets these timing constraints can only be known after the circuit is synthesized. After synthesis is performed, the designer can analyze the circuit to determine whether timing constraints were satisfied using the term **slack**. Slack is the margin by which a timing requirement is met or not met; it is the difference between the required arrival time and the actual arrival time. A positive slack value indicates the margin by which a requirement was met. A negative slack value indicates the margin by which a requirement was not met.

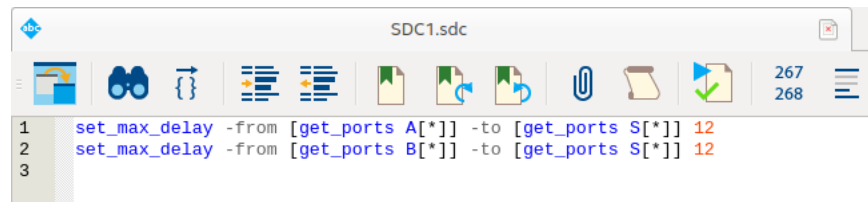
To insert timing constraints in Quartus, select “Synopsys Design Constraints File” from the “File->New” menu.



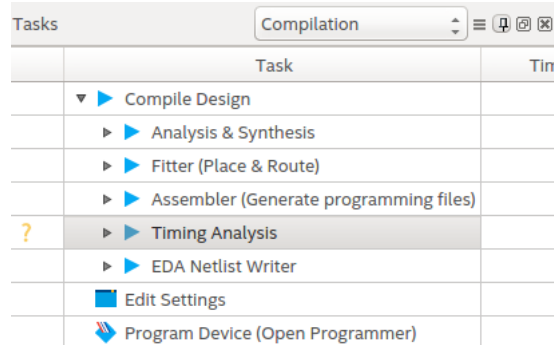
The maximum delay can be specified in the Synopsys Design Constraints File using the following command:

```
set_max_delay -from [get_ports <name_of_input_port>] -to [get_ports <name_of_output_port>] <time in nano seconds>
```

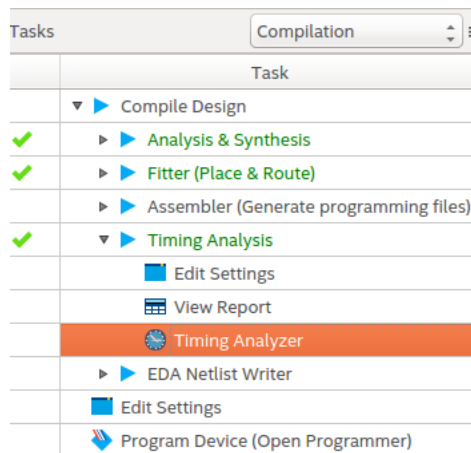
For example, we can specify a maximum delay of 12 ns for all the possible paths from the inputs of the ripple-carry adder to its outputs as shown below.



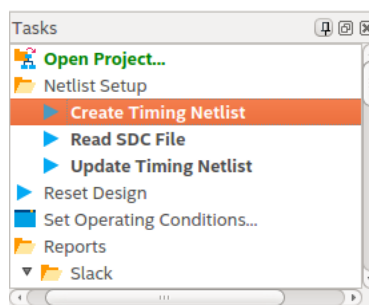
Once the timing constraints are inserted, save the file with the name “firstname_lastname_sdc.sdc”. Recompile your design by double clicking on “Timing Analysis” in the Tasks window of Quartus. *Before recompilation, make sure that the .sdc file is added to the project.*



The Timing Analyzer will read the .sdc file and use the constraint information when performing timing analysis. Once a green check mark appears next to “Timing Analysis”, double click on “Timing Analyzer” under “Timing Analysis” to open the Timing Analyzer tool.

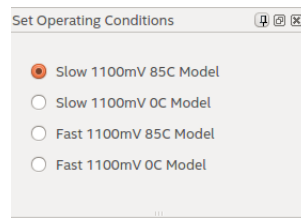


In the Tasks window of the Timing Analyzer tool, double click on “Create Timing Netlist”, “Read SDC File” and “Update Timing Netlist” icons, respectively.

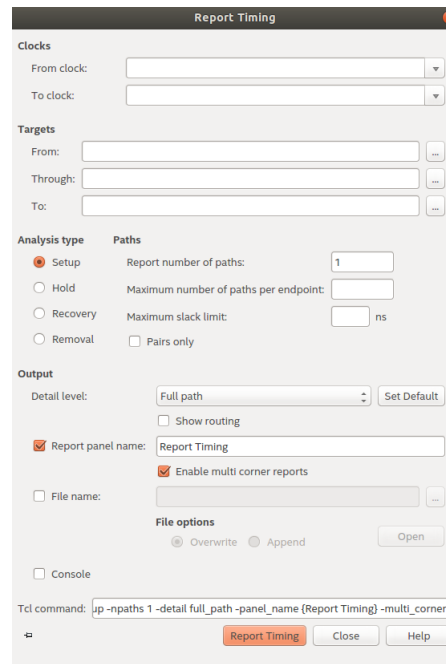


Once completed, the colors of the icons will become green. Before measuring the delay of your design, you should specify the operating conditions of the FPGA device. This can be simply set by selecting one of the possible

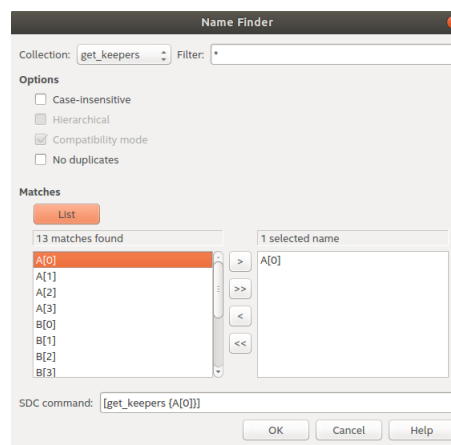
operating conditions listed in the “Set Operating Conditions” in the Timing Analyzer tool.



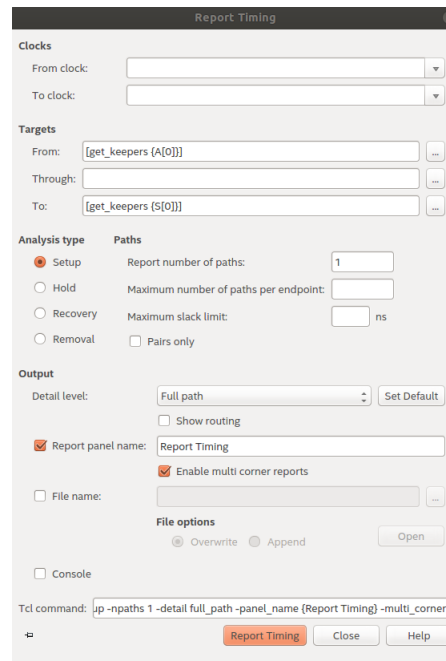
To report the delay of different paths from inputs to outputs, select “Report Timing ...” from the “Reports -> Custom Reports” menu.



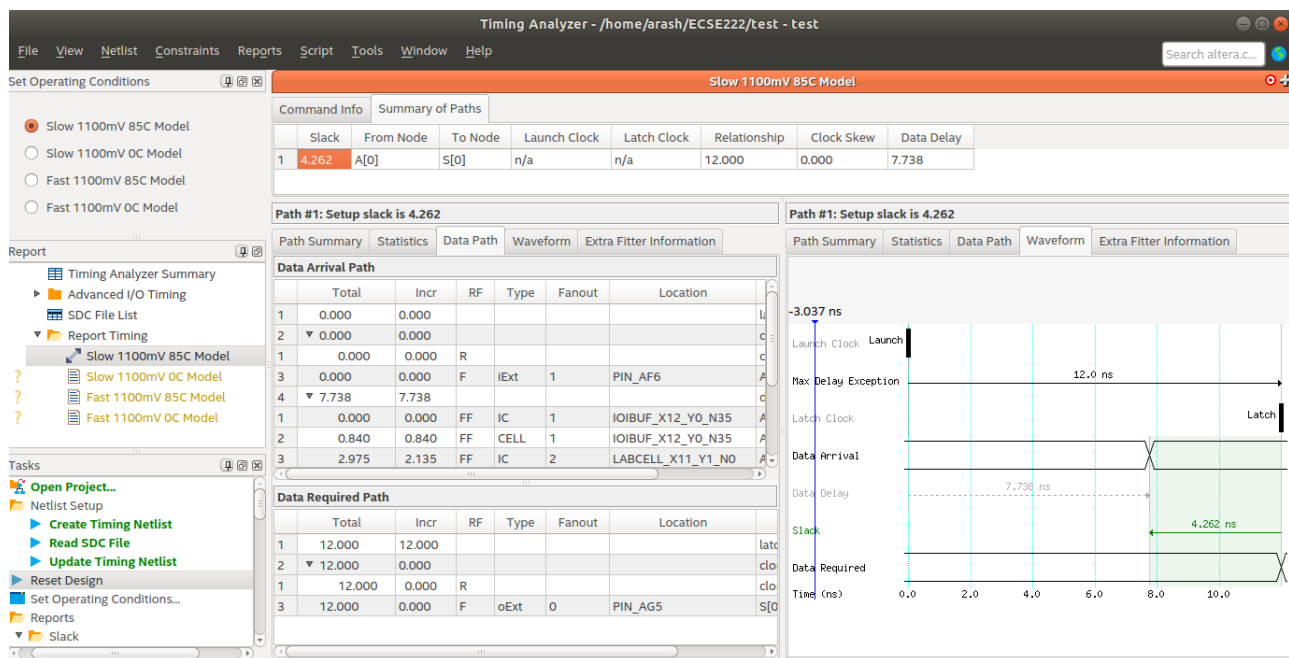
Since no clock signal is associated with your design, we only specify the beginning of the target path(s) by clicking on “From” and the end of the target path/path(s) by clicking on “To” under the section labeled as “Targets”. In the “Name Finder” window that pops up, click on “List” to list all the I/O signals of your design. Select (double click on) a signal (or signals) to determine the beginning of the path that you want to examine and click “OK”. Repeat the same procedure to determine the end of the path.



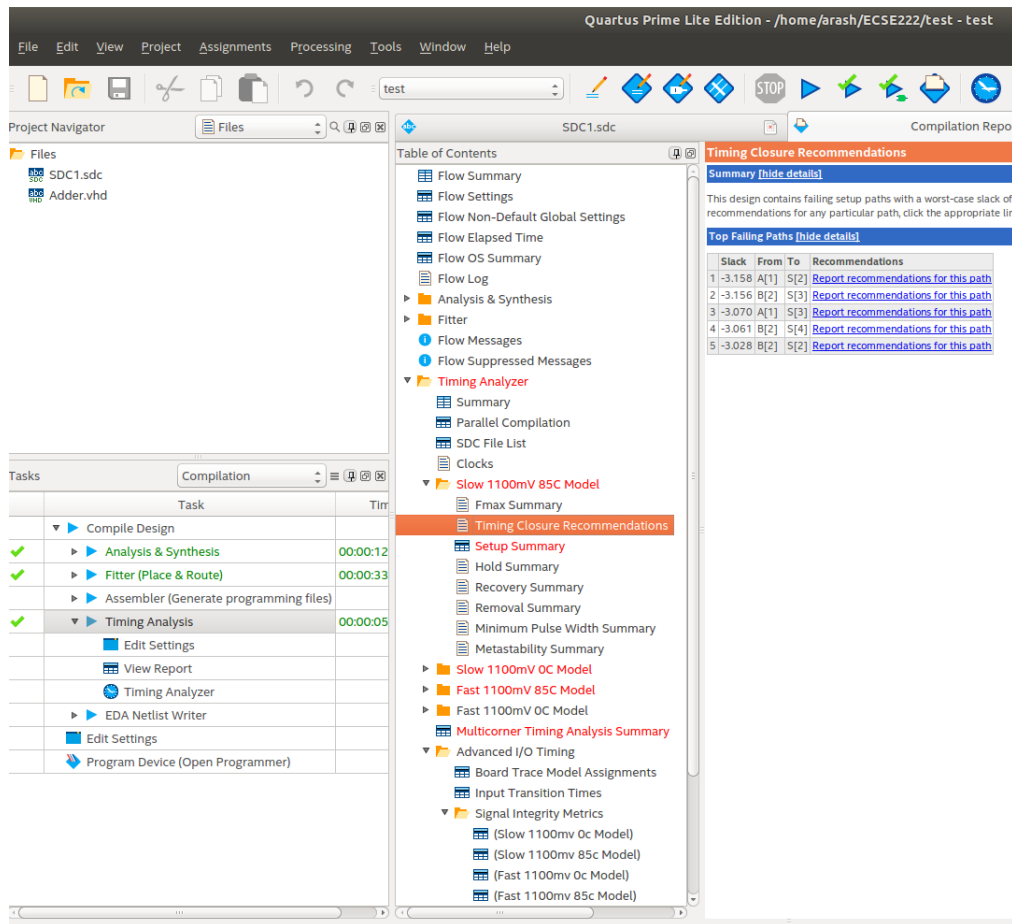
As an example, we can examine the path from the LSB of the input A (*i.e.*, A(0)) to the LSB of the output S (*i.e.*, S(0)) as shown below.



Now, click on “Report Timing” to obtain timing information for the specified path. The delay of the specified path is denoted “Data Delay” in the section entitled “Summary of Paths”. The positive value of the slack denotes the difference between the delay of the path (*i.e.*, Data Delay) and the timing constraint inserted in the .sdc file (*i.e.*, 12 ns). This information is also visualized under the “waveform” tab.



To find the critical path of your design, you should examine all possible paths from all inputs to all outputs and find the one with the longest delay. However, using this “exhaustive search” method is very time consuming as the number of I/O ports increases. To limit the number of paths under examination, we reduce the target delay value in the .sdc file so that a timing violation occurs. For instance, we reduce the target delay constraint of the “circuit under examination” from 12 ns to 5 ns and recompile the design by double clicking on “Timing Analysis” in Quartus. In case of a timing violation, Quartus determines the violating path(s) in the compilation summary of the “Timing Analyzer” tool.



In this approach, our search for the critical path will now be limited to the violating paths. Examining the violating paths in the “Timing Analyzer” tool will, therefore, determine the critical path.

6 Questions

- Briefly explain your VHDL code implementation of all circuits.
- Show representative simulation plots of the half-adder circuit for all possible input values.
- Show representative simulation plots of the full-adder circuit for all possible input values.
- Show representative simulation plots of both behavioral and structural descriptions of the 4-bit RCA for all possible input values.
- Show representative simulation plots of both behavioral and structural descriptions of the one-digit BCD adder circuit for all possible input values.
- Perform timing analysis and find the critical path(s) of the one-digit BCD adder circuit for the Fast 1,100 mV 85C Model. Show the obtained timing waveform(s) of the critical path(s) that you found.
- Report the number of pins and logic modules used to fit your designs on the FPGA board.

	RCA		One-digit BCD adder	
	Structural	Behavioral	Structural	Behavioral
Logic Utilization (in LUTs)				
Total pins				

7 Deliverables

You are required to submit the following deliverables on MyCourses. Please note that a single submission is required per group (by one of the group members).

- Lab report. The report should include the following parts: (1) Names and McGill IDs of group members, (2) an executive summary (short description of what you have done in this VHDL assignment), (3) answers to all questions in previous section (if applicable), (4) legible figures (screenshots) of schematics and simulation results, where all inputs, outputs, signals, and axes are marked and visible, (5) an explanation of the results obtained in the assignments (mark important points on the simulation plots), and (6) conclusions. Note - students are encouraged to take the reports seriously, points will be deducted for sloppy submissions. Please also note that even if some of the waveforms may look the same, you still need to include them separately in the report.
- Project files. Create a single .zip file named `VHDL#_firstname_lastname` (replace # with the number of the current VHDL assignment and `firstname_lastname` with the name of the submitting group member). The .zip file should include the working directory of the project.